Section A - Linked Lists 한국어 번역본

안내: 문제 풀이용 프로그램 템플릿은 APAS 시스템에서 제공됩니다. 함수 구현 시 해당 템플릿을 사용해야 합니다.

1. insertSortedLL

`insertSortedLL()` 함수는 사용자가 입력한 정수를 오름차순이 유지되도록 연결 리스트에 삽입해야 합니다. 현재 연결 리스트 안에 이미 같은 정수가 존재하면 삽입하면 안 됩니다.

함수는 새 항목이 삽입된 인덱스를 반환해야 하며, 작업을 정상적으로 완료하지 못한 경우에는 -1을 반환해야 합니다. 연결 리스트는 비어 있거나 이미 정렬된 상태라고 가정해도 됩니다.

함수 원형: `int insertSortedLL(LinkedList *ll, int item);`

예를 들어 현재 연결 리스트가 2, 3, 5, 7, 9일 때 값 8을 넣으면 결과는 2, 3, 5, 7, 8, 9가 되고, 반환값은 인덱스 4가 됩니다.

현재 연결 리스트가 5, 7, 9, 11, 15일 때 값 7을 넣으면 리스트는 바뀌지 않아야 하며, 이 경우 반환값은 -1입니다.

예시 입출력

```
1: 정렬된 연결 리스트에 정수 삽입
2: 가장 최근에 입력한 값의 인덱스 출력
3: 정렬된 연결 리스트 출력
0: 종료

선택 입력(1/2/3/0): 1
연결 리스트에 추가할 정수 입력: 2
결과 연결 리스트: 2

선택 입력(1/2/3/0): 1
연결 리스트에 추가할 정수 입력: 3
결과 연결 리스트: 2 3

선택 입력(1/2/3/0): 1
연결 리스트에 추가할 정수 입력: 5
결과 연결 리스트: 2 3 5

선택 입력(1/2/3/0): 1
연결 리스트에 추가할 정수 입력: 7
결과 연결 리스트: 2 3 5 7

선택 입력(1/2/3/0): 1
연결 리스트에 추가할 정수 입력: 9
```

```
결과 연결 리스트: 2 3 5 7 9

선택 입력(1/2/3/0): 1
연결 리스트에 추가할 정수 입력: 8
결과 연결 리스트: 2 3 5 7 8 9

선택 입력(1/2/3/0): 2
값 8은 인덱스 4에 추가되었습니다.

선택 입력(1/2/3/0): 3
정렬된 연결 리스트 결과: 2 3 5 7 8 9

선택 입력(1/2/3/0): 1
연결 리스트에 추가할 정수 입력: 5
결과 연결 리스트: 2 3 5 7 8 9

선택 입력(1/2/3/0): 2
값 5는 인덱스 -1에 추가되었습니다.

선택 입력(1/2/3/0): 1
연결 리스트에 추가할 정수 입력: 11
결과 연결 리스트: 2 3 5 7 8 9 11

선택 입력(1/2/3/0): 2
값 11은 인덱스 6에 추가되었습니다.
```

2. alternateMergeLL

`alternateMergeLL()` 함수는 두 번째 리스트의 노드들을 첫 번째 리스트의 번갈아 가는 위치에 삽입해야 합니다. 첫 번째 리스트에 번갈아 끼워 넣을 자리가 있을 때만 두 번째 리스트의 노드를 삽입합니다.

함수 원형: `void alternateMergeLL(LinkedList *ll1, LinkedList *ll2);`

예를 들어 `LinkedList1 = 1, 2, 3`, `LinkedList2 = 4, 5, 6, 7`이면 결과는 `LinkedList1 = 1, 4, 2, 5, 3, 6`, `LinkedList2 = 7`이 됩니다.

첫 번째 리스트가 더 길면 두 번째 리스트는 비게 되어야 합니다. 예를 들어 `LinkedList1 = 1, 5, 7, 3, 9, 11`, `LinkedList2 = 6, 10, 2, 4`라면 결과는 `LinkedList1 = 1, 6, 5, 10, 7, 2, 3, 4, 9, 11`, `LinkedList2 = empty`입니다.

예시 입출력

```
연결 리스트 1: 1 2 3
연결 리스트 2: 4 5 6 7
선택 입력(1/2/3/0): 3
주어진 연결 리스트를 합친 결과:
```

```
연결 리스트 1: 1 4 2 5 3 6
연결 리스트 2: 7
```

3. moveOddItemsToBackLL

moveOddItemsToBackLL() 함수는 연결 리스트 안의 홀수들을 모두 뒤쪽으로 이동시켜야 합니다.

함수 원형: void moveOddItemsToBackLL(LinkedList *ll);

예시

```
리스트가 2, 3, 4, 7, 15, 18 이면 결과는 2 4 18 3 7 15
리스트가 2, 7, 18, 3, 4, 15 이면 결과는 2 18 4 7 3 15
리스트가 1, 3, 5 이면 결과는 1 3 5
리스트가 2, 4, 6 이면 결과는 2 4 6
```

4. moveEvenItemsToBackLL

moveEvenItemsToBackLL() 함수는 연결 리스트 안의 짝수들을 모두 뒤쪽으로 이동시켜야 합니다.

함수 원형: void moveEvenItemsToBackLL(LinkedList *ll);

예시

```
리스트가 2, 3, 4, 7, 15, 18 이면 결과는 3 7 15 2 4 18
리스트가 2, 7, 18, 3, 4, 15 이면 결과는 7 3 15 2 18 4
리스트가 1, 3, 5 이면 결과는 1 3 5
리스트가 2, 4, 6 이면 결과는 2 4 6
```

5. frontBackSplitLL

frontBackSplitLL() 함수는 단일 연결 리스트를 앞쪽 절반과 뒤쪽 절반, 두 개의 서브리스트로 나눠야 합니다. 원소 개수가 홀수이면 남는 하나는 앞쪽 리스트에 들어가야 합니다.

이 함수는 frontList와 backList 두 리스트를 출력합니다.

함수 원형: void frontBackSplitLL(LinkedList *ll, LinkedList *resultFrontList, LinkedList *resultBackList);

예를 들어 리스트가 2, 3, 5, 6, 7이면 결과는 frontList: 2, 3, 5, backList: 6, 7입니다.

예시 입출력

```
1: 연결 리스트에 정수 삽입
2: 연결 리스트 출력
```

```
3: 연결 리스트를 frontList 와 backList 로 분리
0: 종료

선택 입력(1/2/3/0): 1
연결 리스트에 추가할 정수 입력: 2
결과 연결 리스트: 2

선택 입력(1/2/3/0): 1
연결 리스트에 추가할 정수 입력: 3
결과 연결 리스트: 2 3

선택 입력(1/2/3/0): 1
연결 리스트에 추가할 정수 입력: 5
결과 연결 리스트: 2 3 5

선택 입력(1/2/3/0): 1
연결 리스트에 추가할 정수 입력: 6
결과 연결 리스트: 2 3 5 6

선택 입력(1/2/3/0): 1
연결 리스트에 추가할 정수 입력: 7
결과 연결 리스트: 2 3 5 6 7

선택 입력(1/2/3/0): 2
결과 연결 리스트: 2 3 5 6 7

선택 입력(1/2/3/0): 3
주어진 리스트를 분리한 결과:
앞쪽 연결 리스트: 2 3 5
뒤쪽 연결 리스트: 6 7
```

6. moveMaxToFront

`moveMaxToFront()` 함수는 연결 리스트를 최대 한 번만 순회해서, 가장 큰 값을 저장한 노드를 리스트 맨 앞으로 이동시켜야 합니다.

함수 원형: `int moveMaxToFront(ListNode **ptrHead);`

예를 들어 리스트가 (30, 20, 40, 70, 50)이면 결과 리스트는 (70, 30, 20, 40, 50)입니다.

예시 입출력

```
1: 연결 리스트에 정수 삽입
2: 가장 큰 값을 가진 노드를 맨 앞으로 이동
0: 종료
```

```
선택 입력(1/2/0): 1
연결 리스트에 추가할 정수 입력: 30
연결 리스트: 30

선택 입력(1/2/0): 1
연결 리스트에 추가할 정수 입력: 20
연결 리스트: 30 20

선택 입력(1/2/0): 1
연결 리스트에 추가할 정수 입력: 40
연결 리스트: 30 20 40

선택 입력(1/2/0): 1
연결 리스트에 추가할 정수 입력: 70
연결 리스트: 30 20 40 70

선택 입력(1/2/0): 1
연결 리스트에 추가할 정수 입력: 50
연결 리스트: 30 20 40 70 50

선택 입력(1/2/0): 2
결과 연결 리스트: 70 30 20 40 50
```

7. recursiveReverse

`recursiveReverse()` 함수는 재귀적으로 연결 리스트를 뒤집어야 하며, 이를 위해 각 노드의 `next` 포인터와 헤드 포인터를 변경해야 합니다.

함수 원형: `void recursiveReverse(ListNode **ptrHead);`

예를 들어 리스트가 `(1, 2, 3, 4, 5)`이면 결과 리스트는 `(5, 4, 3, 2, 1)`입니다.

예시 입출력

```
1: 연결 리스트에 정수 삽입
2: 뒤집힌 연결 리스트 출력
0: 종료

선택 입력(1/2/0): 1
연결 리스트에 추가할 정수 입력: 1
결과 연결 리스트: 1

선택 입력(1/2/0): 1
연결 리스트에 추가할 정수 입력: 2
```

```
결과 연결 리스트: 1 2

선택 입력(1/2/0): 1
연결 리스트에 추가할 정수 입력: 3
결과 연결 리스트: 1 2 3

선택 입력(1/2/0): 1
연결 리스트에 추가할 정수 입력: 4
결과 연결 리스트: 1 2 3 4

선택 입력(1/2/0): 1
연결 리스트에 추가할 정수 입력: 5
결과 연결 리스트: 1 2 3 4 5

선택 입력(1/2/0): 2
원소를 뒤집은 뒤 결과 연결 리스트: 5 4 3 2 1
```

> 원본 PDF에는 본문 옆에 손글씨 또는 별도 주석처럼 보이는 판독 어려운 문구가 일부 포함되어 있습니다.
> 이번 번역본은 본문에서 명확히 읽히는 문제 설명과 예시 입출력을 중심으로 옮겼습니다.